

AuditPlatform

Rapport d'analyse — laravel-project

Généré le 10/04/2026 à 11:55

■ Informations du projet

Nom du projet	laravel-project
URL du projet	yosrchiha01-group/laravel-project
Branche analysée	main
Date de l'analyse	10/04/2026 11:41
Modèle LLM	llama-3.3-70b-versatile
Analyse OWASP	Activée

■ Scores

Qualité	78	78/100
Sécurité	78	78/100
Performance	55	55/100

■■ Vulnérabilités (226)

A01 - BROKEN ACCESS CONTROL
Sévérité : HAUTE
Fichier : app/Http/Controllers/Admin/AvailabilityCrudController.php — ligne 1
Correction : Ajoutez une authentification avant d'accéder à l'endpoint /availability
A01 - BROKEN ACCESS CONTROL
Sévérité : HAUTE
Fichier : app/Http/Controllers/Admin/ServicesCrudController.php — ligne 1

Correction : Ajoutez une authentification avant d'accéder à l'endpoint /services

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : app/Http/Controllers/Auth/LoginController.php — ligne 1

Correction : Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : app/Http/Controllers/ProfileController.php — ligne 1

Correction : Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : app/Http/Middleware/CheckIfAdmin.php — ligne 1

Correction : Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : app/Http/Requests/AvailabilityRequest.php — ligne 1

Correction : Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : app/Http/Requests/ServicesRequest.php — ligne 1

Correction : Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe

Gestion erreurs

Sévérité : HAUTE

Fichier : app/Http/Controllers/Auth/LoginController.php — ligne 14

Correction : Il est préférable de gérer les exceptions de manière plus spécifique plutôt que d'utiliser des exceptions génériques comme Exception ou Error.

A01 - BROKEN ACCESS CONTROL

Sévérité : HAUTE

Fichier : config/backpack/crud.php — ligne 1

Correction : Vérifiez que les endpoints ne sont pas accessibles sans authentification. Utilisez des méthodes de vérification d'identité robustes.

A10 - SSRF (SERVER-SIDE REQUEST FORGERY)

Sévérité : HAUTE

Fichier : config/backpack/operations/list.php — ligne 1

Correction : Vérifiez que les appels HTTP vers des URLs contrôlées par l'utilisateur sont sécurisés. Utilisez des listes blanches de domaines autorisés.

Chargement de fichiers volumineux entiers en mémoire

Sévérité : HAUTE

Fichier : config/cache.php — ligne 1

Correction : Utiliser des mécanismes de pagination pour charger les données en mémoire

Gestion erreurs

Sévérité : HAUTE

Fichier : config/cache.php — ligne 1

Correction : La classe Illuminate\Cache\CacheManager est chargée de gérer les erreurs de cache. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

Gestion erreurs

Sévérité : HAUTE

Fichier : config/database.php — ligne 1

Correction : La classe Illuminate\Database\Connection est chargée de gérer les erreurs de base de données. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

Gestion erreurs

Sévérité : HAUTE

Fichier : config/filesystems.php — ligne 1

Correction : La classe Illuminate\Filesystem\Filesystem est chargée de gérer les erreurs de fichiers. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

Gestion erreurs

Sévérité : HAUTE

Fichier : config/gravatar.php — ligne 1

Correction : La classe Gravatar est chargée de gérer les erreurs de Gravatar. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

Gestion erreurs

Sévérité : HAUTE

Fichier : config/logging.php — ligne 1

Correction : La classe Monolog\Handler\NullHandler est chargée de gérer les erreurs de logging. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : database/factories/UserFactory.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : database/migrations/0001_01_01_000000_create_users_table.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : database/migrations/2025_01_22_091504_create_availabilities_table.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : database/migrations/2025_01_23_210537_create_reservations_table.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : database/migrations/2025_01_27_212209_create_reviews_table.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : database/migrations/2025_11_28_202930_add_user_id_to_services_table.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : database/seeder/DatabaseSeeder.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : resources/views/admin/profile/show.blade.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : resources/views/admin/questions.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : resources/views/admin/reservation.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : resources/views/ask_question.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : resources/views/auth/login.blade.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : resources/views/layouts/app.blade.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : resources/views/layouts/profile.blade.php — ligne 1

Correction : Utiliser un hashage sécurisé pour les mots de passe, par exemple bcrypt.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : resources/views/questions/index.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : resources/views/reservation.form.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

A08 - DATA INTEGRITY FAILURES

Sévérité : HAUTE

Fichier : ressources/views/reservation/form.blade.php — ligne 1

Correction : Utiliser des contraintes de validation pour les données entrantes.

SOLID-L

Sévérité : HAUTE

Fichier : database/migrations/2025_01_22_091504_create_availabilities_table.php — ligne 12

Correction : La classe Availabilities a un comportement incohérent. La méthode `opening_time()` est définie comme une méthode de classe, mais elle n'est pas utilisée dans la classe. Il est préférable de la supprimer ou de la réécrire pour qu'elle corresponde au comportement attendu.

SOLID-L

Sévérité : HAUTE

Fichier : database/migrations/2025_01_23_210537_create_reservations_table.php — ligne 12

Correction : La classe Reservations a un comportement incohérent. La méthode `reservation_date()` est définie comme une méthode de classe, mais elle n'est pas utilisée dans la classe. Il est préférable de la supprimer ou de la réécrire pour qu'elle corresponde au comportement attendu.

SOLID-L

Sévérité : HAUTE

Fichier : database/migrations/2025_01_27_212209_create_reviews_table.php — ligne 12

Correction : La classe Reviews a un comportement incohérent. La méthode `rating()` est définie comme une méthode de classe, mais elle n'est pas utilisée dans la classe. Il est préférable de la supprimer ou de la réécrire pour qu'elle corresponde au comportement attendu.

SOLID-L

Sévérité : HAUTE

Fichier : database/migrations/2025_11_28_202930_add_user_id_to_services_table.php — ligne 12

Correction : La classe Services a un comportement incohérent. La méthode user_id() est définie comme une méthode de classe, mais elle n'est pas utilisée dans la classe. Il est préférable de la supprimer ou de la réécrire pour qu'elle corresponde au comportement attendu.

SOLID-L

Sévérité : HAUTE

Fichier : database/seeder/DatabaseSeeder.php — ligne 10

Correction : La classe DatabaseSeeder a un comportement incohérent. La méthode run() est définie comme une méthode de classe, mais elle n'est pas utilisée dans la classe. Il est préférable de la supprimer ou de la réécrire pour qu'elle corresponde au comportement attendu.

Maintenabilité

Sévérité : HAUTE

Fichier : resources/views/reservation/list.blade.php — ligne 0

Correction : La fonction 'getReservations' est trop complexe et fait plusieurs choses à la fois. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : HAUTE

Fichier : resources/views/service/show.blade.php — ligne 0

Correction : La fonction 'getReviews' est trop complexe et fait plusieurs choses à la fois. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : HAUTE

Fichier : resources/views/servicesList.blade.php — ligne 0

Correction : La fonction 'getServices' est trop complexe et fait plusieurs choses à la fois. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : HAUTE

Fichier : routes/web.php — ligne 0

Correction : La fonction 'getReservations' est trop complexe et fait plusieurs choses à la fois. Il est préférable de la décomposer en plusieurs fonctions plus petites et plus faciles à comprendre.

A01 - BROKEN ACCESS CONTROL

Sévérité : HAUTE

Fichier : resources/views/welcome.blade.php — ligne 1

Correction : Vérifiez que les endpoints sont protégés par authentification et autorisation appropriées.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : resources/views/service/review.blade.php — ligne 1

Correction : Vérifiez que les tokens JWT sont correctement configurés et protégés contre les attaques de brute force.

A07 - IDENTIFICATION AND AUTHENTICATION FAILURES

Sévérité : HAUTE

Fichier : routes/web.php — ligne 1

Correction : Vérifiez que les routes sont protégées par authentification et autorisation appropriées.

SOLID-O

Sévérité : HAUTE

Fichier : resources/views/service/show.blade.php — ligne 34

Correction : La classe Service est ouverte/closée. Il est préférable de la fermer en utilisant l'héritage ou les interfaces pour éviter de la modifier à l'avenir.

Complexité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/Admin/AvailabilityCrudController.php — ligne 0

Correction : Réduire la complexité de la méthode setup() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/Auth/LoginController.php — ligne 0

Correction : Réduire la complexité de la méthode login() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ProfileController.php — ligne 0

Correction : Réduire la complexité de la méthode update() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ReservationController.php — ligne 0

Correction : Réduire la complexité de la méthode updateStatus() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ReviewController.php — ligne 0

Correction : Réduire la complexité de la méthode store() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ServiceController.php — ligne 0

Correction : Réduire la complexité de la méthode show() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Http/Middleware/CheckIfAdmin.php — ligne 0

Correction : Réduire la complexité de la méthode `handle()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : `app/Http/Requests/AvailabilityRequest.php` — ligne 0

Correction : Réduire la complexité de la méthode `authorize()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : `app/Http/Requests/ServicesRequest.php` — ligne 0

Correction : Réduire la complexité de la méthode `authorize()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : `app/Models/Availability.php` — ligne 0

Correction : Réduire la complexité de la méthode `user()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : `app/Models/Question.php` — ligne 0

Correction : Réduire la complexité de la méthode `user()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : `app/Models/Reservation.php` — ligne 0

Correction : Réduire la complexité de la méthode `service()` en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Models/Review.php — ligne 0

Correction : Réduire la complexité de la méthode service() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Models/Service.php — ligne 0

Correction : Réduire la complexité de la méthode reviews() en utilisant des méthodes plus petites et plus faciles à comprendre.

Maintenabilité

Sévérité : MOYENNE

Fichier : app/Models/User.php — ligne 0

Correction : Réduire la complexité de la méthode services() en utilisant des méthodes plus petites et plus faciles à comprendre.

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Http/Controllers/QuestionController.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ReservationController.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ReviewController.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/Availability.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/Question.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/Reservation.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/Review.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/Service.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : app/Models/User.php — ligne 1

Correction : Utilisez des méthodes de désérialisation sécurisées pour les données entrantes

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Http/Controllers/Admin/AvailabilityCrudController.php — ligne 23

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `setupListOperation()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/Admin/ServicesCrudController.php` — ligne 23

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `setupListOperation()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/ProfileController.php` — ligne 15

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `show()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/QuestionController.php` — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `index()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/ReservationController.php` — ligne 15

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `updateStatus()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/ReviewController.php` — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `create()`

Docstring manquant

Sévérité : MOYENNE

Fichier : `app/Http/Controllers/ServiceController.php` — ligne 15

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `show()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Http/Middleware/CheckIfAdmin.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `checkIfUserIsAdmin()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Http/Requests/AvailabilityRequest.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `authorize()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Http/Requests/ServicesRequest.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `authorize()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/Availability.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `user()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/Question.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `user()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/Reservation.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode `service()`

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/Review.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode service()

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/Service.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode reviews()

Docstring manquant

Sévérité : MOYENNE

Fichier : app/Models/User.php — ligne 10

Correction : Ajoutez une docstring pour expliquer le rôle de la méthode services()

SOLID-S

Sévérité : MOYENNE

Fichier : app/Http/Controllers/Admin/AvailabilityCrudController.php — ligne 24

Correction : La méthode setup() contient plusieurs responsabilités. Il est préférable de la diviser en plusieurs méthodes plus petites et plus faciles à comprendre.

SOLID-S

Sévérité : MOYENNE

Fichier : app/Http/Controllers/Admin/ServicesCrudController.php — ligne 24

Correction : La méthode setup() contient plusieurs responsabilités. Il est préférable de la diviser en plusieurs méthodes plus petites et plus faciles à comprendre.

Testabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ProfileController.php — ligne 24

Correction : La méthode update() contient des effets de bord cachés. Il est préférable de mocker ces effets de bord pour faciliter les tests.

Testabilité

Sévérité : MOYENNE

Fichier : app/Http/Controllers/ReservationController.php — ligne 24

Correction : La méthode `updateStatus()` contient des effets de bord cachés. Il est préférable de mocker ces effets de bord pour faciliter les tests.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/operations/list.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/operations/list.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/operations/reorder.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/operations/reorder.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/operations/show.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/operations/show.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Complexité

Sévérité : MOYENNE

Fichier : config/backpack/operations/update.php — ligne 0

Correction : Réduire la complexité de la fonction en utilisant des fonctions plus petites et plus faciles à comprendre.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/operations/update.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/theme-tabler.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/backpack/ui.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/cache.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/database.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/filesystems.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/gravatar.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

Lisibilité

Sévérité : MOYENNE

Fichier : config/logging.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables temporelles.

A04 - INSECURE DESIGN

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 1

Correction : Vérifiez que la logique métier ne peut pas être contournée par un utilisateur malveillant. Utilisez des méthodes de validation côté serveur.

A05 - SECURITY MISCONFIGURATION

Sévérité : MOYENNE

Fichier : config/backpack/theme-tabler.php — ligne 1

Correction : Vérifiez que le mode debug n'est pas activé en production. Utilisez des en-têtes HTTP de sécurité.

A09 - LOGGING AND MONITORING FAILURES

Sévérité : MOYENNE

Fichier : config/logging.php — ligne 1

Correction : Vérifiez que les logs ne contiennent pas de données sensibles. Utilisez des méthodes de suppression de données sensibles.

SELECT*

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 1

Correction : Utiliser des requêtes SELECT spécifiques aux colonnes nécessaires au lieu de SELECT *

Requêtes N+1

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 1

Correction : Utiliser des jointures pour réduire le nombre de requêtes

Boucle $O(n^2)$

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 1

Correction : Utiliser des boucles imbriquées avec complexité $O(n)$ ou pire sans justification

Recherche linéaire $O(n)$

Sévérité : MOYENNE

Fichier : config/backpack/operations/list.php — ligne 1

Correction : Utiliser des index/dictionnaires pour améliorer les performances

Boucle $O(n^2)$

Sévérité : MOYENNE

Fichier : config/backpack/operations/update.php — ligne 1

Correction : Utiliser des boucles imbriquées avec complexité $O(n)$ ou pire sans justification

Concaténation de chaînes en boucle

Sévérité : MOYENNE

Fichier : config/backpack/theme-tabler.php — ligne 1

Correction : Utiliser StringBuilder pour améliorer les performances

Absence d'index sur les colonnes utilisées dans WHERE, JOIN, ORDER BY

Sévérité : MOYENNE

Fichier : config/database.php — ligne 1

Correction : Créer des index sur les colonnes utilisées dans les requêtes

Fichiers ouverts sans bloc try/finally ou gestionnaire de contexte

Sévérité : MOYENNE

Fichier : config/filesystems.php — ligne 1

Correction : Utiliser des blocs try/finally ou des gestionnaires de contexte pour fermer les fichiers

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 1

Correction : Ajouter une docstring pour décrire les préférences de CRUD

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'opération de création

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/form.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de la forme

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/list.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'opération de liste

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/reorder.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'opération de repositionnement

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/show.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'opération de visualisation

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/operations/update.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'opération de mise à jour

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/theme-tablet.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations du thème

Docstring manquant

Sévérité : MOYENNE

Fichier : config/backpack/ui.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de l'interface utilisateur

Docstring manquant

Sévérité : MOYENNE

Fichier : config/cache.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de la cache

Docstring manquant

Sévérité : MOYENNE

Fichier : config/database.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de la base de données

Docstring manquant

Sévérité : MOYENNE

Fichier : config/filesystems.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations des systèmes de fichiers

Docstring manquant

Sévérité : MOYENNE

Fichier : config/gravatar.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de Gravatar

Docstring manquant

Sévérité : MOYENNE

Fichier : config/logging.php — ligne 1

Correction : Ajouter une docstring pour décrire les configurations de la journalisation

SOLID-S

Sévérité : MOYENNE

Fichier : config/backpack/crud.php — ligne 1

Correction : La classe Backpack\CRUD::preferences est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/create.php — ligne 1

Correction : La classe Backpack\CRUD::CreateOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/form.php — ligne 1

Correction : La classe Backpack\CRUD::FormOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/list.php — ligne 1

Correction : La classe Backpack\CRUD::ListOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/reorder.php — ligne 1

Correction : La classe Backpack\CRUD::ReorderOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/show.php — ligne 1

Correction : La classe Backpack\CRUD::ShowOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/operations/update.php — ligne 1

Correction : La classe Backpack\CRUD::UpdateOperation est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/theme-tabler.php — ligne 1

Correction : La classe Backpack\Theme::Tabler est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

SOLID-O

Sévérité : MOYENNE

Fichier : config/backpack/ui.php — ligne 1

Correction : La classe Backpack\UI est chargée de plusieurs responsabilités. Il est préférable de la diviser en plusieurs classes plus petites et plus spécialisées.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/factories/UserFactory.php — ligne 0

Correction : La méthode definition() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_21_122846_create_services_table.php — ligne 0

Correction : La méthode up() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_22_091504_create_availabilities_table.php — ligne 0

Correction : La méthode up() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_23_210537_create_reservations_table.php — ligne 0

Correction : La méthode up() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_27_212209_create_reviews_table.php — ligne 0

Correction : La méthode up() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/migrations/2025_11_28_202930_add_user_id_to_services_table.php — ligne 0

Correction : La méthode up() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Maintenabilité

Sévérité : MOYENNE

Fichier : database/seeders/DatabaseSeeder.php — ligne 0

Correction : La méthode run() contient 10 chemins d'exécution, ce qui dépasse la limite de 10. Il est recommandé de réorganiser la logique pour réduire la complexité.

Boucle O(n)

Sévérité : MOYENNE

Fichier : database/factories/UserFactory.php — ligne 15

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

SELECT *

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_21_122846_create_services_table.php — ligne 14

Correction : Sélectionner uniquement les colonnes nécessaires

Boucle O(n)

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_22_091504_create_availabilities_table.php — ligne 10

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_23_210537_create_reservations_table.php — ligne 10

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_27_212209_create_reviews_table.php — ligne 10

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : database/migrations/2025_11_28_202930_add_user_id_to_services_table.php — ligne 10

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : resources/views/admin/profile/show.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : resources/views/admin/questions.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : resources/views/admin/reservation.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/ask_question.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/auth/login.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/chooseRegion.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/layouts/app.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/layouts/profile.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : ressources/views/questions/index.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle $O(n)$

Sévérité : MOYENNE

Fichier : resources/views/reservation.form.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Boucle O(n)

Sévérité : MOYENNE

Fichier : resources/views/reservation/form.blade.php — ligne 25

Correction : Utiliser un index ou un dictionnaire pour accéder aux éléments de la collection

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/factories/UserFactory.php — ligne 17

Correction : Le nom de la variable \$password devrait être plus explicite, par exemple \$passwordHash

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_21_122846_create_services_table.php — ligne 10

Correction : Le nom de la colonne 'title' devrait être plus explicite, par exemple 'nom_service'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_22_085441_add_phone_address_profile_picture_to_users_table.php — ligne 5

Correction : Le nom des colonnes 'phone', 'address' et 'profile_picture' devraient être plus explicites, par exemple 'numero_telephone', 'adresse_complete' et 'image_de_profil'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_22_091504_create_availabilities_table.php — ligne 10

Correction : Le nom de la colonne 'daily_availability' devrait être plus explicite, par exemple 'disponibilite_journee'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_23_192834_create_services_table.php — ligne 10

Correction : Le nom de la colonne 'user_id' devrait être plus explicite, par exemple 'id_utilisateur'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_23_210537_create_reservations_table.php — ligne 10

Correction : Le nom de la colonne 'service_id' devrait être plus explicite, par exemple 'id_service'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_01_27_212209_create_reviews_table.php — ligne 10

Correction : Le nom de la colonne 'service_id' devrait être plus explicite, par exemple 'id_service'

Nommage ambigu

Sévérité : MOYENNE

Fichier : database/migrations/2025_11_28_202930_add_user_id_to_services_table.php — ligne 10

Correction : Le nom de la colonne 'user_id' devrait être plus explicite, par exemple 'id_utilisateur'

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/admin/profile/show.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition `if(Auth::check())`

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/admin/reservation.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if (\$reservations->isEmpty())

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/ask_question.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if(session('success'))

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/auth/login.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if(\$errors->any())

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/layouts/app.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if(Auth::check())

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/layouts/profile.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if(Auth::check())

Commentaire absent

Sévérité : MOYENNE

Fichier : resources/views/reservation.form.blade.php — ligne 24

Correction : Il manque un commentaire pour expliquer la logique de la condition if(\$service->user->name)

SOLID-S

Sévérité : MOYENNE

Fichier : database/factories/UserFactory.php — ligne 13

Correction : La méthode definition() contient plusieurs responsabilités. Il est préférable de la diviser en plusieurs méthodes plus petites et plus faciles à comprendre.

Gestion erreurs

Sévérité : MOYENNE

Fichier : resources/views/admin/profile/show.blade.php — ligne 24

Correction : La variable \$user n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : resources/views/admin/questions.blade.php — ligne 24

Correction : La variable \$questions n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : resources/views/admin/reservation.blade.php — ligne 24

Correction : La variable \$reservations n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : resources/views/ask_question.blade.php — ligne 24

Correction : La variable \$questions n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : resources/views/auth/login.blade.php — ligne 24

Correction : La variable \$errors n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : ressources/views/layouts/app.blade.php — ligne 24

Correction : La variable \$errors n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : ressources/views/layouts/profile.blade.php — ligne 24

Correction : La variable \$errors n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : ressources/views/questions/index.blade.php — ligne 24

Correction : La variable \$questions n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : ressources/views/reservation.form.blade.php — ligne 24

Correction : La variable \$service n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Gestion erreurs

Sévérité : MOYENNE

Fichier : ressources/views/reservation/form.blade.php — ligne 24

Correction : La variable \$service n'est pas définie dans le contexte de la vue. Il est préférable de la définir avant d'utiliser la variable dans la vue.

Lisibilité

Sévérité : MOYENNE

Fichier : ressources/views/reservation/forme.blade.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme par exemple 'nomUtilisateur' au lieu de 'name'.

Lisibilité

Sévérité : MOYENNE

Fichier : ressources/views/service/review.blade.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme par exemple 'noteUtilisateur' au lieu de 'rating'.

Lisibilité

Sévérité : MOYENNE

Fichier : ressources/views/serviceDetails.blade.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme par exemple 'disponibilitePrestataire' au lieu de 'availability'.

Lisibilité

Sévérité : MOYENNE

Fichier : ressources/views/welcome.blade.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs, comme par exemple 'rechercheUtilisateur' au lieu de 'searchInput'.

A03 - INJECTION

Sévérité : MOYENNE

Fichier : ressources/views/welcome.blade.php — ligne 1

Correction : Vérifiez que les données utilisateur sont correctement échappées avant d'être rendues dans les templates.

A08 - DATA INTEGRITY FAILURES

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 1

Correction : Vérifiez que les données utilisateur sont correctement validées et sécurisées avant d'être stockées dans la base de données.

A09 - LOGGING AND MONITORING FAILURES

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 1

Correction : Vérifiez que les logs sont correctement configurés et collectés pour détecter les anomalies et les attaques.

A05 - SECURITY MISCONFIGURATION

Sévérité : MOYENNE

Fichier : tailwind.config.js — ligne 1

Correction : Vérifiez que les configurations de Tailwind sont correctes et ne contiennent pas de vulnérabilités de sécurité.

SELECT*

Sévérité : MOYENNE

Fichier : resources/views/reservation/list.blade.php — ligne 17

Correction : Utiliser les colonnes nécessaires au lieu de SELECT *

Requêtes N+1

Sévérité : MOYENNE

Fichier : resources/views/reservation/list.blade.php — ligne 17

Correction : Utiliser des jointures pour réduire le nombre de requêtes

Boucles imbriquées

Sévérité : MOYENNE

Fichier : resources/views/serviceDetails.blade.php — ligne 64

Correction : Utiliser des boucles plus efficaces pour améliorer les performances

Boucles imbriquées

Sévérité : MOYENNE

Fichier : resources/views/servicesList.blade.php — ligne 24

Correction : Utiliser des boucles plus efficaces pour améliorer les performances

Appels API externes à l'intérieur de boucles

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 34

Correction : Utiliser des mécanismes de cache pour améliorer les performances

Absence de timeout sur les appels HTTP

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 34

Correction : Utiliser des timeouts pour éviter les blocages

Absence de mécanisme de cache pour des données rarement modifiées

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 34

Correction : Utiliser des mécanismes de cache pour améliorer les performances

Docstring manquant

Sévérité : MOYENNE

Fichier : ressources/views/welcome.blade.php — ligne 0

Correction : Ajouter une description de la page d'accueil

Commentaire absent

Sévérité : MOYENNE

Fichier : ressources/views/service/review.blade.php — ligne 0

Correction : Ajouter des commentaires pour expliquer les choix de design et les fonctionnalités

Docstring manquant

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 0

Correction : Ajouter des commentaires pour expliquer les routes et les fonctionnalités associées

Commentaire absent

Sévérité : MOYENNE

Fichier : routes/console.php — ligne 0

Correction : Ajouter des commentaires pour expliquer les commandes et les fonctionnalités associées

SOLID-S

Sévérité : MOYENNE

Fichier : resources/views/service/review.blade.php — ligne 14

Correction : La méthode review() contient plusieurs responsabilités. Il est préférable de la décomposer en plusieurs méthodes plus petites et plus faciles à tester.

Gestion erreurs

Sévérité : MOYENNE

Fichier : routes/web.php — ligne 34

Correction : La méthode storeReservation() ne gère pas les erreurs de validation. Il est préférable d'utiliser try/catch pour gérer les exceptions et de retourner un message d'erreur à l'utilisateur.

Lisibilité

Sévérité : FAIBLE

Fichier : app/Http/Controllers/Admin/ServicesCrudController.php — ligne 0

Correction : Utiliser des noms de variables plus significatifs et éviter les variables avec moins de 2 caractères significatifs.

Commentaire absent

Sévérité : FAIBLE

Fichier : app/Http/Controllers/Auth/LoginController.php — ligne 10

Correction : Ajoutez un commentaire pour expliquer la logique de la méthode showLoginForm()

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/admin/profile/show.blade.php — ligne 0

Correction : Le nom de la variable \$user n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/admin/reservation.blade.php — ligne 0

Correction : Le nom de la variable \$reservations n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/ask_question.blade.php — ligne 0

Correction : Le nom de la variable \$questions n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/layouts/app.blade.php — ligne 0

Correction : Le nom de la variable \$profile n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/layouts/profile.blade.php — ligne 0

Correction : Le nom de la variable \$profile n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : resources/views/reservation.form.blade.php — ligne 0

Correction : Le nom de la variable \$service n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Lisibilité

Sévérité : FAIBLE

Fichier : ressources/views/reservation/form.blade.php — ligne 0

Correction : Le nom de la variable \$service n'est pas explicite. Il est recommandé de renommer la variable pour améliorer la lisibilité.

Concaténation de chaînes en boucle

Sévérité : FAIBLE

Fichier : ressources/views/service/review.blade.php — ligne 14

Correction : Utiliser un StringBuilder pour améliorer les performances

Nommage ambigu

Sévérité : FAIBLE

Fichier : ressources/views/serviceDetails.blade.php — ligne 0

Correction : Utiliser des noms de variables plus explicites, comme par exemple `disponibilite` au lieu de `availability`

Testabilité

Sévérité : FAIBLE

Fichier : tests/Feature/ExampleTest.php — ligne 10

Correction : La méthode `test_the_application_returns_a_successful_response()` n'est pas suffisamment testée. Il est préférable d'utiliser des tests unitaires et d'intégration pour couvrir toutes les fonctionnalités de l'application.

■ Recommandations

1. Réduire la complexité des méthodes

Réduire la complexité des méthodes en utilisant des méthodes plus petites et plus faciles à comprendre.

2. Améliorer la lisibilité des variables

Utiliser des noms de variables plus significatifs et éviter les variables avec moins de 2 caractères significatifs.

3. Améliorer la maintenabilité des classes

Réduire la complexité des classes en utilisant des classes plus petites et plus faciles à comprendre.

4. Utilisez des méthodes de désérialisation sécurisées

Utilisez des méthodes de désérialisation sécurisées pour les données entrantes pour éviter les attaques de code injection

5. Utilisez un algorithme de hachage sécurisé

Utilisez un algorithme de hachage sécurisé pour stocker les mots de passe pour éviter les attaques de force brute

6. Ajouter des docstrings pour expliquer les rôles des méthodes

Ajoutez des docstrings pour expliquer les rôles des méthodes dans les classes et les contrôleurs

7. Ajouter des commentaires pour expliquer la logique des méthodes

Ajoutez des commentaires pour expliquer la logique des méthodes dans les classes et les contrôleurs

8. Utiliser des méthodes plus petites et plus faciles à comprendre

Divisez les méthodes `setup()` dans les contrôleurs `AvailabilityCrudController` et `ServicesCrudController` en plusieurs méthodes plus petites et plus faciles à comprendre.

9. Gérer les exceptions de manière plus spécifique

Utilisez des exceptions plus spécifiques plutôt que des exceptions génériques comme `Exception` ou `Error` dans le contrôleur `LoginController`.

10. Mocker les effets de bord cachés

Mockez les effets de bord cachés dans les méthodes `update()` du contrôleur `ProfileController` et `updateStatus()` du contrôleur `ReservationController` pour faciliter les tests.

11. Réduire la complexité des fonctions

Les fonctions doivent être petites et faciles à comprendre. Il est recommandé de les diviser en plusieurs fonctions plus petites et plus faciles à comprendre.

12. Utiliser des noms de variables significatifs

Les noms de variables doivent être significatifs et éviter les variables temporelles. Cela facilite la compréhension et la maintenance du code.

13. Utiliser des commentaires

Les commentaires doivent être utilisés pour expliquer le code et faciliter la compréhension de celui-ci.

14. Utiliser des fonctions de validation

Les fonctions de validation doivent être utilisées pour vérifier les données et éviter les erreurs.

15. Authentification robuste

Utilisez des méthodes de vérification d'identité robustes pour empêcher les accès non autorisés.

16. Validation côté serveur

Utilisez des méthodes de validation côté serveur pour empêcher les contournements de la logique métier.

17. Sécurisation des appels HTTP

Utilisez des listes blanches de domaines autorisés pour empêcher les appels HTTP vers des URLs contrôlées par l'utilisateur.

18. Sécurisation du mode debug

Utilisez des en-têtes HTTP de sécurité pour empêcher l'accès non autorisé au mode debug.

19. Suppression de données sensibles

Utilisez des méthodes de suppression de données sensibles pour empêcher la révélation de données sensibles dans les logs.

20. Utiliser des requêtes SELECT spécifiques aux colonnes nécessaires

Utiliser des requêtes SELECT spécifiques aux colonnes nécessaires au lieu de `SELECT *` pour améliorer les performances.

21. Utiliser des jointures pour réduire le nombre de requêtes

Utiliser des jointures pour réduire le nombre de requêtes et améliorer les performances.

22. Utiliser des boucles imbriquées avec complexité $O(n)$ ou pire sans justification

Utiliser des boucles imbriquées avec complexité $O(n)$ ou pire sans justification pour améliorer les performances.

23. Utiliser des index/dictionnaires pour améliorer les performances

Utiliser des index/dictionnaires pour améliorer les performances et réduire le temps de recherche.

24. Utiliser des mécanismes de pagination pour charger les données en mémoire

Utiliser des mécanismes de pagination pour charger les données en mémoire et améliorer les performances.

25. Créer des index sur les colonnes utilisées dans les requêtes

Créer des index sur les colonnes utilisées dans les requêtes pour améliorer les performances.

26. Utiliser des blocs try/finally ou des gestionnaires de contexte pour fermer les fichiers

Utiliser des blocs try/finally ou des gestionnaires de contexte pour fermer les fichiers et améliorer les performances.

27. Ajouter des docstrings

Ajouter des docstrings pour décrire les préférences de CRUD, les configurations de l'opération de création, les configurations de la forme, les configurations de l'opération de liste, les configurations de l'opération de repositionnement, les configurations de l'opération de visualisation, les configurations de l'opération de mise à jour, les configurations du thème, les configurations de l'interface utilisateur, les configurations de la cache, les configurations de la base de données, les configurations des systèmes de fichiers, les configurations de Gravatar et les configurations de la journalisation.

28. Diviser les classes en plusieurs classes plus petites et plus spécialisées

Les classes chargées de plusieurs responsabilités sont difficiles à maintenir et à comprendre. Il est préférable de les diviser en plusieurs classes plus petites et plus spécialisées.

29. Utiliser les exceptions spécifiques

Les exceptions génériques (Exception, Error) sont difficiles à gérer et à comprendre. Il est préférable d'utiliser des exceptions spécifiques pour chaque type d'erreur.

30. Gérer les erreurs de manière efficace

Les erreurs non gérées peuvent entraîner des problèmes de sécurité et de stabilité. Il est préférable de gérer les erreurs de manière efficace en utilisant des exceptions spécifiques et des mécanismes de gestion d'erreurs.

31. Réorganiser la logique pour réduire la complexité

Il est recommandé de réorganiser la logique pour réduire la complexité et améliorer la maintenabilité du code.

32. Améliorer la lisibilité

Il est recommandé de renommer les variables pour améliorer la lisibilité et faciliter la compréhension du code.

33. Utiliser des hashages sécurisés pour les mots de passe

Utiliser des hashages sécurisés comme bcrypt pour les mots de passe pour éviter les attaques par force brute.

34. Utiliser des contraintes de validation pour les données entrantes

Utiliser des contraintes de validation pour les données entrantes pour éviter les injections SQL et les attaques par données sensibles.

35. Utiliser des index pour améliorer les performances

Les index peuvent considérablement améliorer les performances de vos requêtes SQL. Assurez-vous de les utiliser pour les colonnes qui sont souvent utilisées dans vos requêtes.

36. Utiliser des dictionnaires pour accéder aux éléments de collections

Les dictionnaires peuvent être utilisés pour accéder aux éléments de collections de manière efficace. Assurez-vous de les utiliser pour les collections qui sont souvent utilisées dans vos applications.

37. Optimiser les boucles

Les boucles peuvent être optimisées pour améliorer les performances de vos applications. Assurez-vous de les utiliser de manière efficace et d'éviter les boucles imbriquées.

38. Ajouter des commentaires explicatifs

Il est recommandé d'ajouter des commentaires explicatifs pour expliquer la logique de chaque fonction, méthode et condition dans le code.

39. Améliorer la nommagem des variables et colonnes

Il est recommandé d'améliorer la nommagem des variables et colonnes pour qu'elles soient plus explicites et faciles à comprendre.

40. Améliorer la cohérence des classes

Certaines classes ont des comportements incohérents. Il est préférable de les réécrire pour qu'elles correspondent au comportement attendu.

41. Améliorer la gestion des erreurs

Certaines variables ne sont pas définies dans le contexte de la vue. Il est préférable de les définir avant d'utiliser la variable dans la vue.

42. Utiliser des noms de variables plus significatifs

Les noms de variables devraient être clairs et significatifs pour faciliter la compréhension du code. Il est préférable d'éviter les abréviations et les noms génériques.

43. Décomposer les fonctions complexes

Les fonctions devraient être petites et faciles à comprendre. Il est préférable de décomposer les fonctions complexes en plusieurs fonctions plus petites et plus faciles à comprendre.

44. Authentification et autorisation

Vérifiez que les endpoints sont protégés par authentification et autorisation appropriées pour éviter les attaques de sécurité.

45. Échappement des données utilisateur

Vérifiez que les données utilisateur sont correctement échappées avant d'être rendues dans les templates pour éviter les attaques de sécurité.

46. Tokens JWT

Vérifiez que les tokens JWT sont correctement configurés et protégés contre les attaques de brute force pour éviter les attaques de sécurité.

47. Validation des données utilisateur

Vérifiez que les données utilisateur sont correctement validées et sécurisées avant d'être stockées dans la base de données pour éviter les attaques de sécurité.

48. Logs et monitoring

Vérifiez que les logs sont correctement configurés et collectés pour détecter les anomalies et les attaques pour éviter les attaques de sécurité.

49. Configurations de Tailwind

Vérifiez que les configurations de Tailwind sont correctes et ne contiennent pas de vulnérabilités de sécurité pour éviter les attaques de sécurité.

50. Utiliser des mécanismes de cache pour améliorer les performances

Les mécanismes de cache permettent de stocker les résultats de requêtes fréquentes pour éviter de les recalculer à chaque fois. Cela peut améliorer les performances en réduisant le nombre de requêtes à la base de données.

51. Utiliser des timeouts pour éviter les blocages

Les timeouts permettent de définir un temps maximum pour les appels HTTP. Si l'appel ne répond pas dans ce temps, le système peut passer à autre chose et éviter les blocages.

52. Améliorer la lisibilité du code

Utiliser des commentaires pour expliquer les choix de design et les fonctionnalités, et utiliser des noms de variables plus explicites

53. Améliorer la documentation

Ajouter des commentaires pour expliquer les routes et les fonctionnalités associées, et ajouter des descriptions de la page d'accueil

54. Utiliser des interfaces pour définir les contrats

Les interfaces permettent de définir les contrats que doivent respecter les classes qui les implémentent. Cela aide à éviter les erreurs de conception et à améliorer la maintenabilité de l'application.

55. Utiliser des tests unitaires et d'intégration

Les tests unitaires et d'intégration permettent de couvrir toutes les fonctionnalités de l'application et de détecter les erreurs de conception. Cela aide à améliorer la qualité et la maintenabilité de l'application.